L03 — Hive: SQL trên HDFS, partitioning, định dạng file, bucketing

Môi trường: Hive 4.2.1 (HiveServer2 + Metastore độc lập, Tez local mode), Metastore DB PostgreSQL 18, HDFS 3.4.3. Dữ liệu: **cả tháng 01/2024** (2,964,624 dòng, CSV 274.2 MB), không dùng mẫu 1 triệu dòng. Log gốc: các file `0*.txt`, `1*.txt` (đọc nhanh bằng `./clean.sh <file>`); file SQL đã hoàn thành nằm trong thư mục này; ảnh ở `img/`.

File SQL đã hoàn thành

File	TODO
`01_external_tables.sql`	TODO 1: `zones` dùng `OpenCSVSerde`, `LOCATION '/data/zones'`, `skip.header.line.count=1`
`02_partitioned_tables.sql`	TODO 2: `hive.exec.dynamic.partition.mode=nonstrict;` TODO 3: `date_format(tpep_pickup_datetime,'yyyy-MM');` TODO 4: `trips_parquet` (Parquet + Snappy)
`04_partition_pruning.sql`	TODO 5: `EXPLAIN DEPENDENCY` với filter trên `tpep_pickup_datetime`
`05_bucketing.sql`	TODO 6: `CLUSTERED BY (pulocationid) SORTED BY (pulocationid) INTO 4 BUCKETS`

sql/01_external_tables.sql (đã hoàn thành)

```
-- L03 step 2: external tables over files that already sit in HDFS.
-- Hive only stores METADATA (schema + location) in the metastore; DROP TABLE keeps the files.
CREATE DATABASE IF NOT EXISTS taxi;
USE taxi;

DROP TABLE IF EXISTS trips_csv;
CREATE EXTERNAL TABLE trips_csv (
  vendorid              INT,
  tpep_pickup_datetime  TIMESTAMP,
  tpep_dropoff_datetime TIMESTAMP,
  passenger_count       BIGINT,
  trip_distance         DOUBLE,
  ratecodeid            BIGINT,
  store_and_fwd_flag    STRING,
  pulocationid          INT,
  dolocationid          INT,
  payment_type          BIGINT,
  fare_amount           DOUBLE,
  extra                 DOUBLE,
  mta_tax               DOUBLE,
  tip_amount            DOUBLE,
  tolls_amount          DOUBLE,
  improvement_surcharge DOUBLE,
  total_amount          DOUBLE,
  congestion_surcharge  DOUBLE,
  airport_fee           DOUBLE
)
ROW FORMAT DELIMITED FIELDS TERMINATED BY ','
STORED AS TEXTFILE
LOCATION '/data/taxi_csv';

-- The zone lookup has a header line and "quoted","values" -> OpenCSVSerde
-- (OpenCSVSerde reads every column as STRING; we cast when querying).
DROP TABLE IF EXISTS zones;
CREATE EXTERNAL TABLE zones (
  locationid   STRING,
  borough      STRING,
  zone         STRING,
  service_zone STRING
)
ROW FORMAT SERDE 'org.apache.hadoop.hive.serde2.OpenCSVSerde'
STORED AS TEXTFILE
LOCATION '/data/zones'
TBLPROPERTIES ('skip.header.line.count' = '1');

SHOW TABLES;
SELECT * FROM trips_csv LIMIT 3;
SELECT * FROM zones LIMIT 3;
SELECT count(*) AS csv_rows FROM trips_csv;
```

sql/02_partitioned_tables.sql (đã hoàn thành)

```
-- L03 step 3: managed, columnar, partitioned copies of the CSV data.
USE taxi;
SET hive.exec.dynamic.partition = true;
-- TODO 2: dynamic partitioning is "strict" by default (at least one static partition value
--         required). Set hive.exec.dynamic.partition.mode so ALL partition values can be dynamic.
SET hive.exec.dynamic.partition.mode = nonstrict;


DROP TABLE IF EXISTS trips_orc;
CREATE TABLE trips_orc (
  vendorid INT, tpep_pickup_datetime TIMESTAMP, tpep_dropoff_datetime TIMESTAMP,
  passenger_count BIGINT, trip_distance DOUBLE, ratecodeid BIGINT, store_and_fwd_flag STRING,
  pulocationid INT, dolocationid INT, payment_type BIGINT, fare_amount DOUBLE, extra DOUBLE,
  mta_tax DOUBLE, tip_amount DOUBLE, tolls_amount DOUBLE, improvement_surcharge DOUBLE,
  total_amount DOUBLE, congestion_surcharge DOUBLE, airport_fee DOUBLE
)
PARTITIONED BY (pickup_month STRING)
STORED AS ORC
TBLPROPERTIES ('orc.compress' = 'ZLIB');

-- Dynamic partitioning: the LAST column of the SELECT feeds the partition column.
INSERT OVERWRITE TABLE trips_orc PARTITION (pickup_month)
SELECT t.*,
       date_format(t.tpep_pickup_datetime, 'yyyy-MM') AS pickup_month
FROM trips_csv t;

-- TODO 4: create trips_parquet: same columns and partitioning as trips_orc, STORED AS PARQUET
--         (TBLPROPERTIES ('parquet.compression' = 'SNAPPY')), and fill it the same way.
DROP TABLE IF EXISTS trips_parquet;
CREATE TABLE trips_parquet (
  vendorid INT, tpep_pickup_datetime TIMESTAMP, tpep_dropoff_datetime TIMESTAMP,
  passenger_count BIGINT, trip_distance DOUBLE, ratecodeid BIGINT, store_and_fwd_flag STRING,
  pulocationid INT, dolocationid INT, payment_type BIGINT, fare_amount DOUBLE, extra DOUBLE,
  mta_tax DOUBLE, tip_amount DOUBLE, tolls_amount DOUBLE, improvement_surcharge DOUBLE,
  total_amount DOUBLE, congestion_surcharge DOUBLE, airport_fee DOUBLE
)
PARTITIONED BY (pickup_month STRING)
STORED AS PARQUET
TBLPROPERTIES ('parquet.compression' = 'SNAPPY');

INSERT OVERWRITE TABLE trips_parquet PARTITION (pickup_month)
SELECT t.*, date_format(t.tpep_pickup_datetime, 'yyyy-MM') AS pickup_month
FROM trips_csv t;


SHOW PARTITIONS trips_orc;
SELECT pickup_month, count(*) AS trips FROM trips_orc GROUP BY pickup_month ORDER BY pickup_month;
```

sql/04_partition_pruning.sql (đã hoàn thành)

```
-- L03 step 5: does Hive read all partitions or only the ones we need?
USE taxi;

-- (a) filter on the PARTITION column -> only 1 partition is an input
EXPLAIN DEPENDENCY
SELECT count(*) FROM trips_orc WHERE pickup_month = '2024-01';

-- (b) filter on a normal column -> every partition is an input
-- TODO 5: write the same count for January 2024, but filter on tpep_pickup_datetime
--         instead of pickup_month, prefixed with EXPLAIN DEPENDENCY. How many partitions are read?
EXPLAIN DEPENDENCY
SELECT count(*) FROM trips_orc
WHERE tpep_pickup_datetime >= '2024-01-01' AND tpep_pickup_datetime < '2024-02-01';


-- (c) the full plan: look for "filterExpr", "Statistics: Num rows" and the partition list
EXPLAIN EXTENDED
SELECT count(*) FROM trips_orc WHERE pickup_month = '2024-01';

-- (d) where does a partition live?
DESCRIBE FORMATTED trips_orc PARTITION (pickup_month = '2024-01');
```

sql/05_bucketing.sql (đã hoàn thành)

```
-- L03 step 6: bucketing = hash(column) mod N -> a fixed number of files per partition/table.
USE taxi;

DROP TABLE IF EXISTS trips_bucketed;
CREATE TABLE trips_bucketed (
  tpep_pickup_datetime TIMESTAMP,
  pulocationid INT,
  dolocationid INT,
  fare_amount DOUBLE,
  total_amount DOUBLE
)
-- TODO 6: bucket by pulocationid into 4 buckets, sorted by pulocationid
CLUSTERED BY (pulocationid) SORTED BY (pulocationid) INTO 4 BUCKETS
STORED AS ORC;

INSERT OVERWRITE TABLE trips_bucketed
SELECT tpep_pickup_datetime, pulocationid, dolocationid, fare_amount, total_amount
FROM trips_orc WHERE pickup_month = '2024-01';

DESCRIBE FORMATTED trips_bucketed;

-- Sampling reads just one bucket (~1/4 of the data):
SELECT count(*) FROM trips_bucketed TABLESAMPLE (BUCKET 1 OUT OF 4 ON pulocationid);
SELECT count(*) FROM trips_bucketed;

-- Every row of zone 132 (JFK) is in exactly one bucket file:
SELECT INPUT__FILE__NAME AS file, count(*) AS rows_for_jfk
FROM trips_bucketed WHERE pulocationid = 132 GROUP BY INPUT__FILE__NAME;
```

1–2. Khởi động stack và đưa CSV lên HDFS

L03 — docker compose ps + CSV lên HDFS

```
$ docker compose ps
NAME                    STATUS                   PORTS
l03_hive-datanode-1     Up 14 seconds
l03_hive-hiveserver2-1  Up 6 seconds             0.0.0.0:10000->10000/tcp, [::]:10000->10000/tcp, 0.0.0.0:10002->10002/tcp, [::]:10002->10002/tcp
l03_hive-metastore-1    Up 6 seconds             0.0.0.0:9083->9083/tcp, [::]:9083->9083/tcp
l03_hive-namenode-1     Up 14 seconds (healthy)  0.0.0.0:9870->9870/tcp, [::]:9870->9870/tcp
l03_hive-postgres-1     Up 14 seconds (healthy)  5432/tcp

$ parquet_to_csv + hdfs dfs -put
wrote 2,964,624 rows -> /datasets/data/taxi/csv/yellow_2024-01.csv (274.2 MB)
Found 1 items
-rw-r--r--   1 root supergroup    261.5 M 2026-10-30 10:30 /data/taxi_csv/yellow_2024-01.csv
Found 1 items
-rw-r--r--   1 root supergroup     12.0 K 2026-10-30 10:30 /data/zones/taxi_zone_lookup.csv
```

Home | Local logs | Metrics Dump | Hive Configuration | Stack Trace | Llap Daemons | Configure logging

HiveServer2

Active Sessions

User Name	IP Address	Operation Count	Active Time (s)	Idle Time (s)

Total number of sessions: 0

Open Queries

User Name	Query	Execution Engine	State	Opened Timestamp	Opened (s)	Latency (s)	Drilldown Link

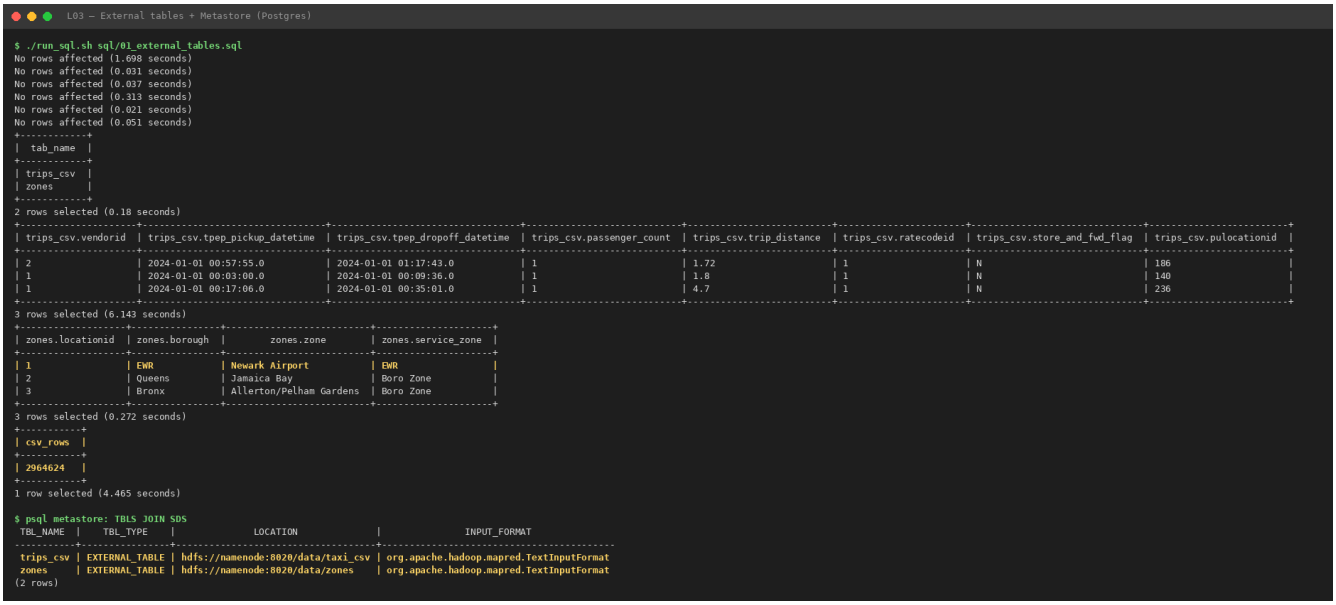
Total number of queries: 0

Last Max 25 Closed Queries

User Name	Query	Execution Engine	State	Opened (s)	Closed Timestamp	Latency (s)	Drilldown Link
hive	SELECT count(*) AS stats_count FROM trips_orc	tez	FINISHED	2	Fri Oct 30 10:36:42 GMT 2026	2	Drilldown
hive	EXPLAIN SELECT count(*) FROM trips_orc	tez	FINISHED	0	Fri Oct 30 10:36:40 GMT 2026	0	Drilldown
hive	USE taxi	tez	FINISHED	0	Fri Oct 30 10:36:40 GMT 2026	0	Drilldown
hive	DESCRIBE FORMATTED zone_notes	tez	FINISHED	0	Fri Oct 30 10:36:22 GMT 2026	0	Drilldown
hive	SELECT * FROM zone_notes ORDER BY locationid	tez	FINISHED	1	Fri Oct 30 10:36:22 GMT 2026	1	Drilldown
hive	DELETE FROM zone_notes WHERE locationid = 161	tez	FINISHED	3	Fri Oct 30 10:36:20 GMT 2026	3	Drilldown
hive	UPDATE zone_notes SET note = 'JFK Airport' WHERE locationid = 132	tez	FINISHED	4	Fri Oct 30 10:36:17 GMT 2026	4	Drilldown
hive	INSERT INTO zone_notes VALUES (132, 'JFK'), (138, 'LaGuardia'), (161, 'Midtown')	tez	FINISHED	3	Fri Oct 30 10:36:13 GMT 2026	3	Drilldown
hive	CREATE TABLE zone_notes (locationid INT, note STRING) STORED AS ORC TBLPROPERTIES ('transactional'='true')	tez	FINISHED	0	Fri Oct 30 10:36:09 GMT 2026	0	Drilldown
hive	DROP TABLE IF EXISTS zone_notes	tez	FINISHED	0	Fri Oct 30 10:36:09 GMT 2026	0	Drilldown
hive	SELECT count(*) AS stats_count FROM trips_orc	tez	FINISHED	1	Fri Oct 30 10:36:09 GMT 2026	1	Drilldown

3. External tables

3 dòng mẫu, 3 zone (`1 | EWR | Newark Airport | EWR...`), `csv_rows = 2,964,624`. Metastore (Postgres) chỉ lưu tên bảng, kiểu bảng, `LOCATION` và `INPUT_FORMAT`:



Hình 1: external + metastore

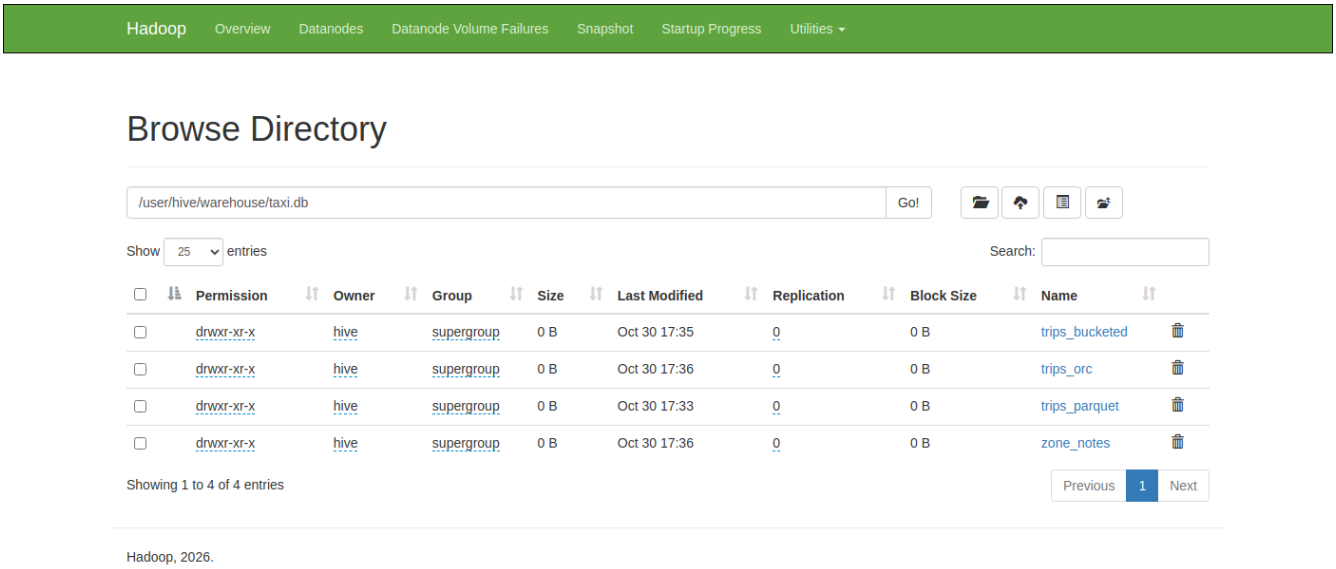
4. Bảng partition ORC và Parquet

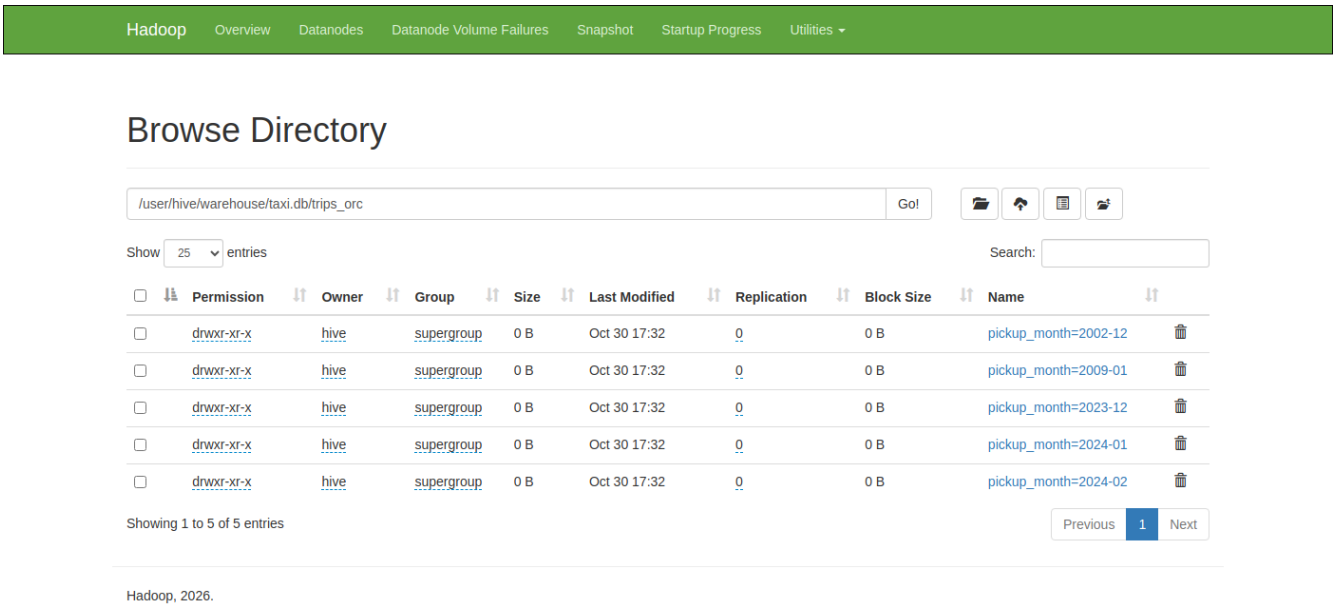
INSERT OVERWRITE: ORC 42.8 s, Parquet 45.1 s (2,964,624 dòng mỗi bảng). `SHOW PARTITIONS` cho **5 partition** (dữ liệu cả tháng có thêm 2024-02 so với mẫu 1 triệu dòng):

pickup_month	trips
2002-12	2
2009-01	3
2023-12	10
2024-01	2,964,606
2024-02	3

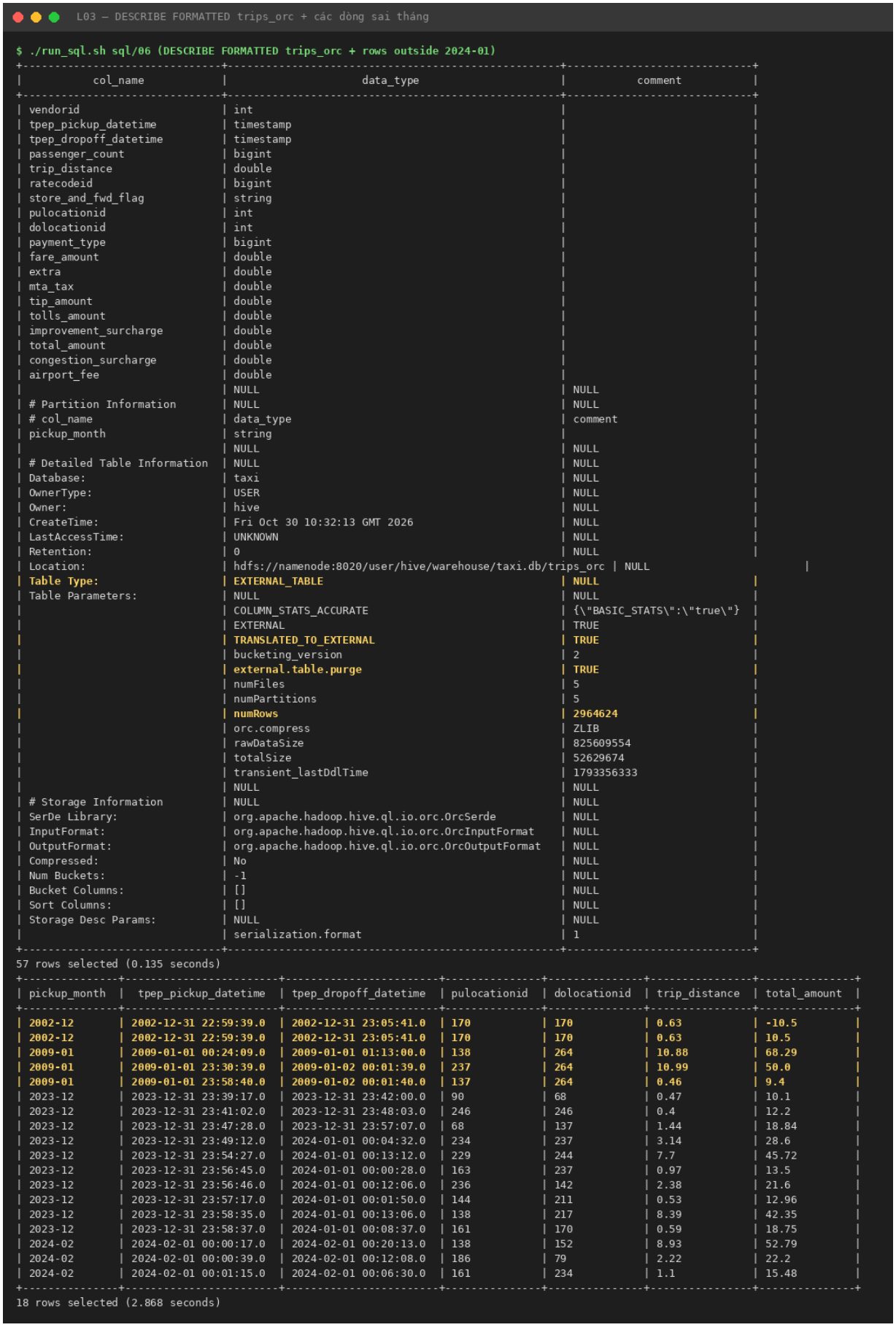
L03 — Dynamic partitioning ORC + Parquet

```
$ ./run_sql.sh sql/02_partitioned_tables.sql
No rows affected (0.06 seconds)
No rows affected (0.006 seconds)
No rows affected (0.004 seconds)
No rows affected (0.017 seconds)
No rows affected (0.07 seconds)
2,964,624 rows affected (42.761 seconds)
No rows affected (0.017 seconds)
No rows affected (0.052 seconds)
2,964,624 rows affected (45.134 seconds)
+------------------------+
|       partition        |
+------------------------+
| pickup_month=2002-12   |
| pickup_month=2009-01   |
| pickup_month=2023-12   |
| pickup_month=2024-01   |
| pickup_month=2024-02   |
+------------------------+
5 rows selected (0.077 seconds)
+---------------+----------+
| pickup_month  |  trips   |
+---------------+----------+
| 2002-12       | 2        |
| 2009-01       | 3        |
| 2023-12       | 10       |
| 2024-01       | 2964606  |
| 2024-02       | 3        |
+---------------+----------+
5 rows selected (2.24 seconds)
```





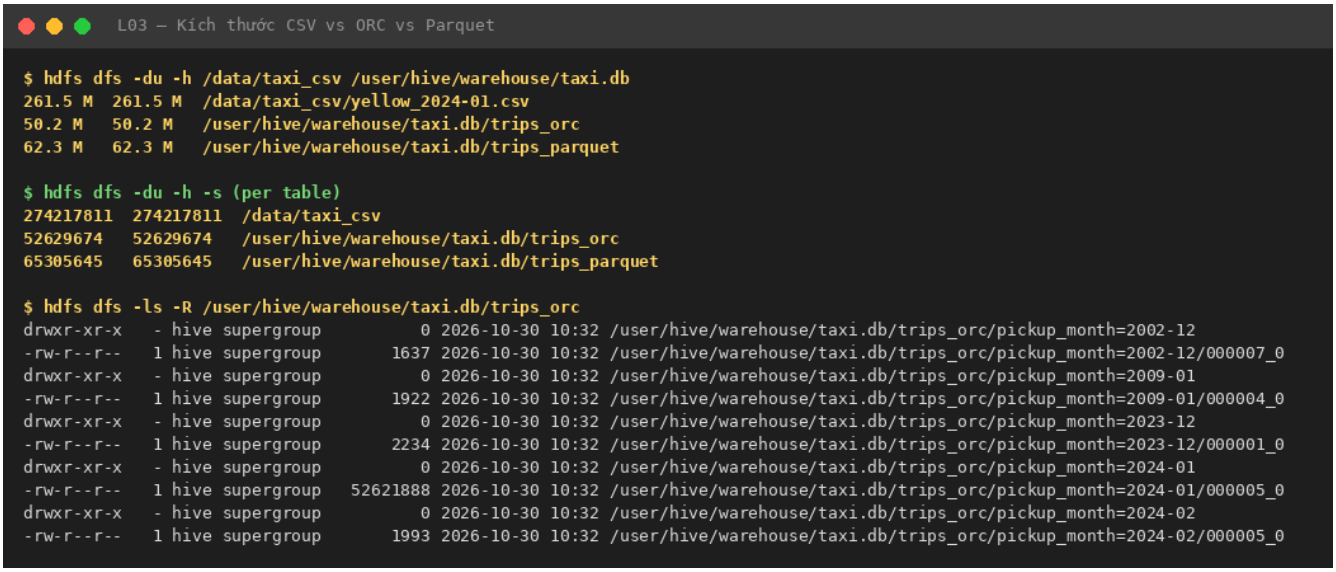
`DESCRIBE FORMATTED trips_orc` cho thấy `Table Type: EXTERNAL_TABLE`, `TRANSLATED_TO_EXTERNAL=TRUE`, `external.table.purge=TRUE` (đúng hành vi Hive 4 với bảng "managed" không phải ACID).



Hình 2: describe + bad rows

Bảng kích thước

Định dạng	Kích thước (cả tháng, 2.96 triệu dòng)	So với CSV
CSV (text)	274,217,811 B (261.5 MiB)	1×
ORC + ZLIB	52,629,674 B (50.2 MiB)	**5.2× nhỏ hơn**
Parquet + Snappy	65,305,645 B (62.3 MiB)	**4.2× nhỏ hơn**
ORC bucketed (chỉ 5 cột)	25,271,072 B (24.1 MiB)	10.9×

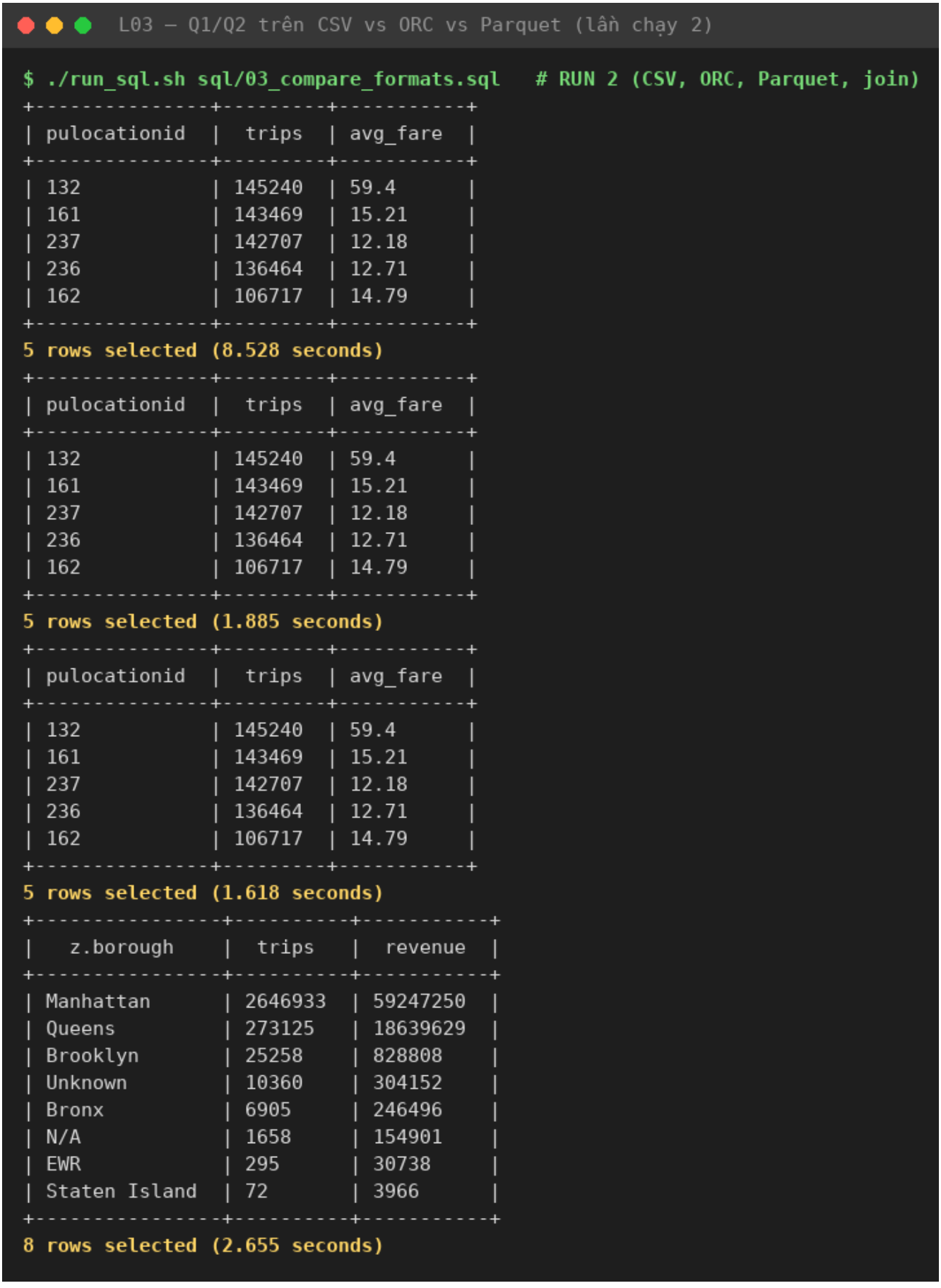


Hình 3: sizes

5. CSV, ORC và Parquet: bảng thời gian (lần chạy thứ 2)

Query	CSV	ORC	Parquet
Q1 top pickup zones tháng 01/2024	**8.53 s**	**1.89 s**	**1.62 s**
Q2 join zones theo borough (ORC)		2.66 s	

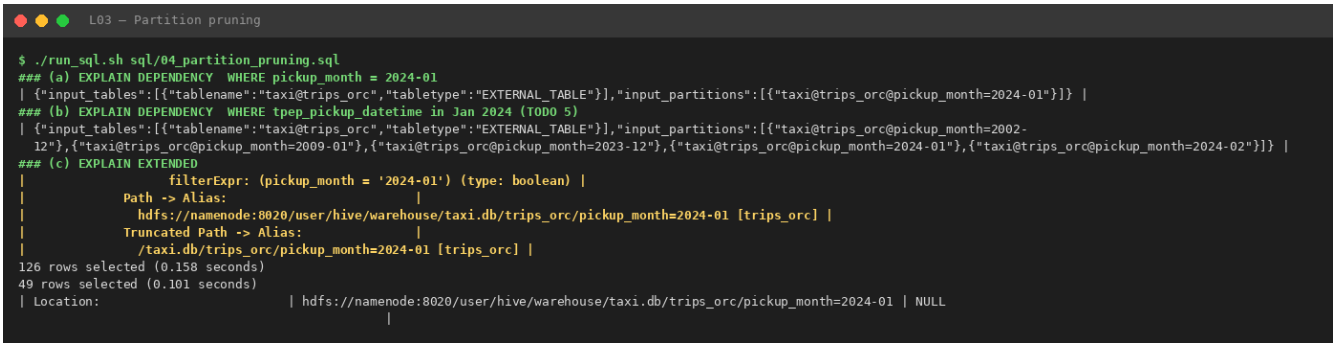
(Lần chạy 1: 9.22 / 1.99 / 1.77 s.) Q1 dòng đầu: zone 132 (JFK) có **145,240 chuyến**, avg fare **59.40**; ba nguồn cho kết quả giống hệt nhau. Q2: Manhattan 2,646,933 chuyến (**89.3 %**), Queens 273,125, Brooklyn 25,258…



Hình 4: compare

6. Partition pruning

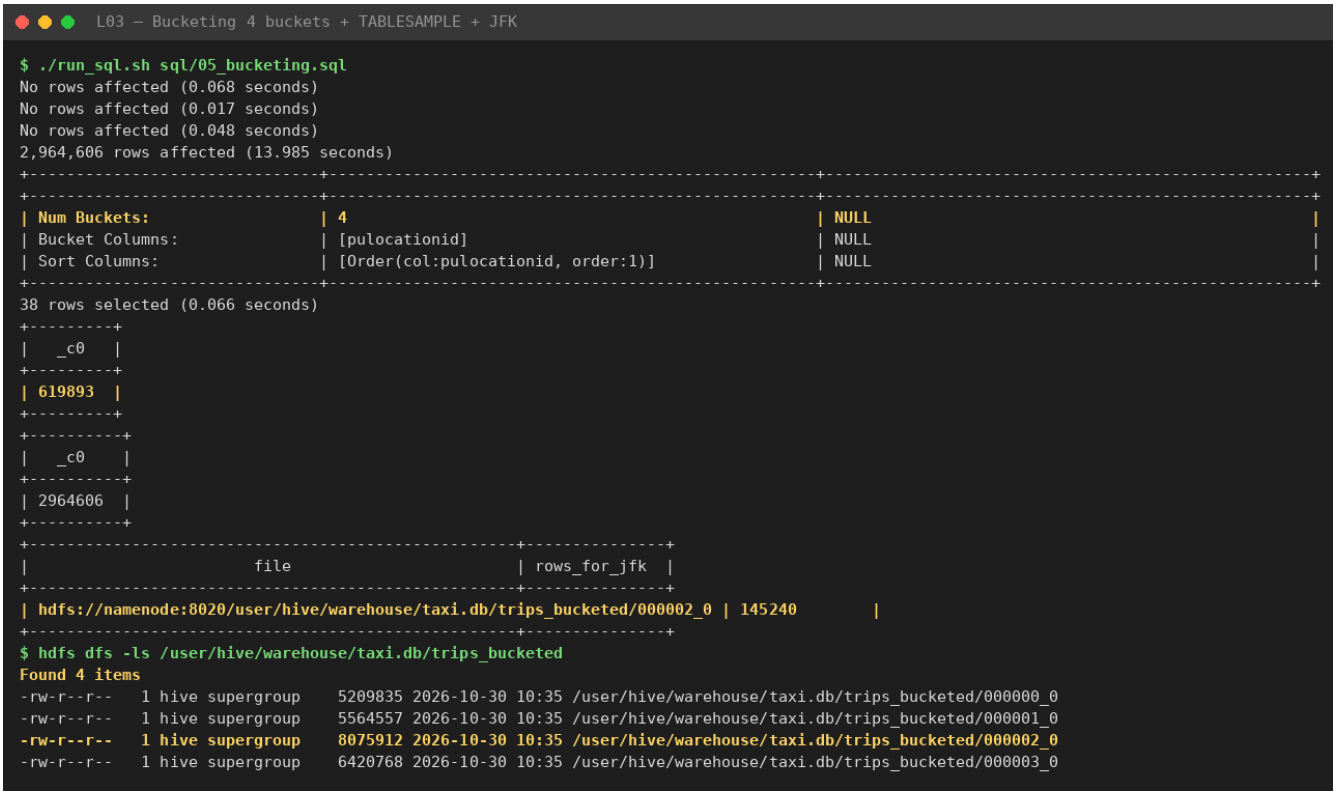
- (a) `WHERE pickup_month = '2024-01'` → `input_partitions` chỉ có **1** partition.
- (b) TODO 5, `WHERE tpep_pickup_datetime` trong tháng 1 → **cả 5** partition là input (Hive không suy được partition từ cột thường).
- (c) `EXPLAIN EXTENDED: filterExpr: (pickup_month = '2024-01'), Path -> Alias` chỉ có `.../trips_orc/pickup_month=2024-01`.

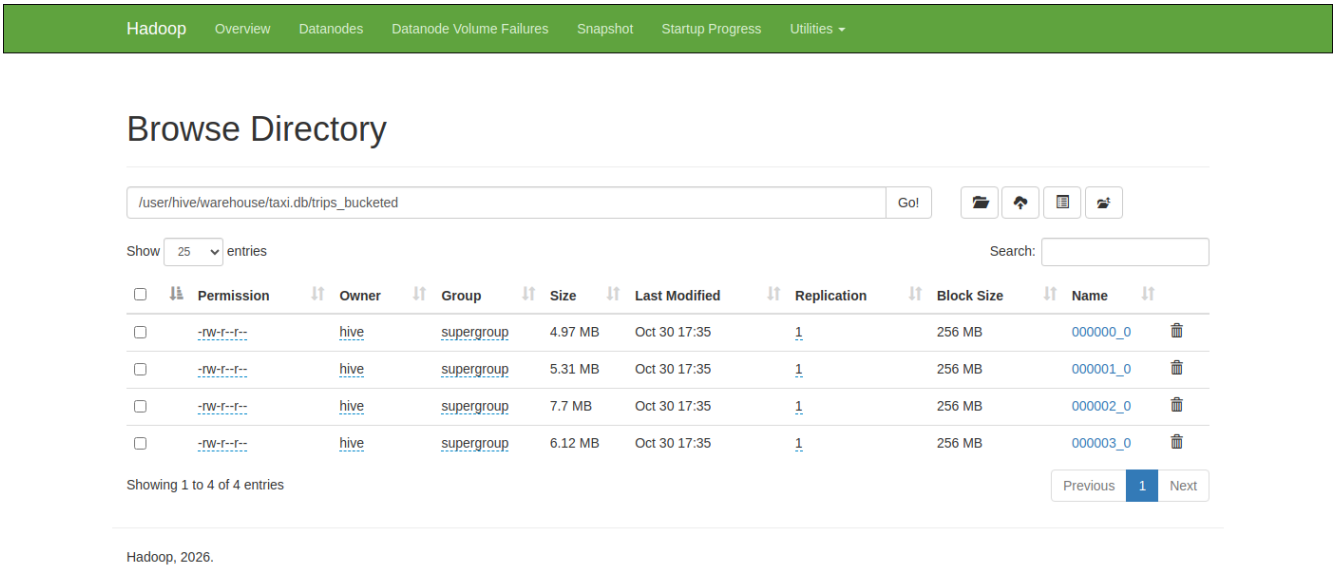


Hình 5: pruning

7. Bucketing

Đúng **4 file** `000000_0 … 000003_0`. `TABLESAMPLE(BUCKET 1 OUT OF 4)` = 619,893 dòng (~21 %, vì bucket theo hash nên không đều). Cả **145,240 dòng JFK** đều nằm trong **một** file `000002_0` (vì vậy file này lớn nhất, 8.1 MB).

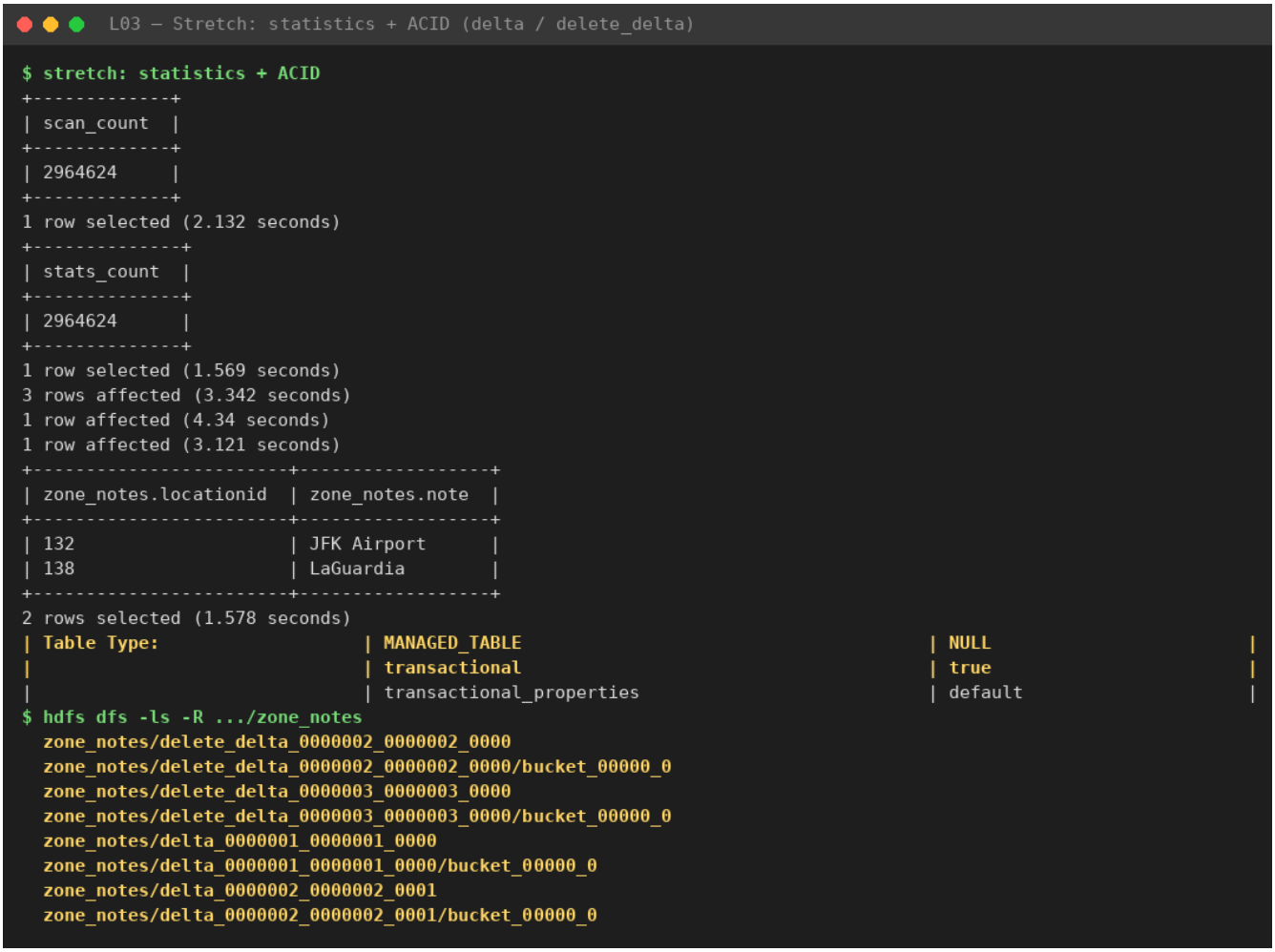


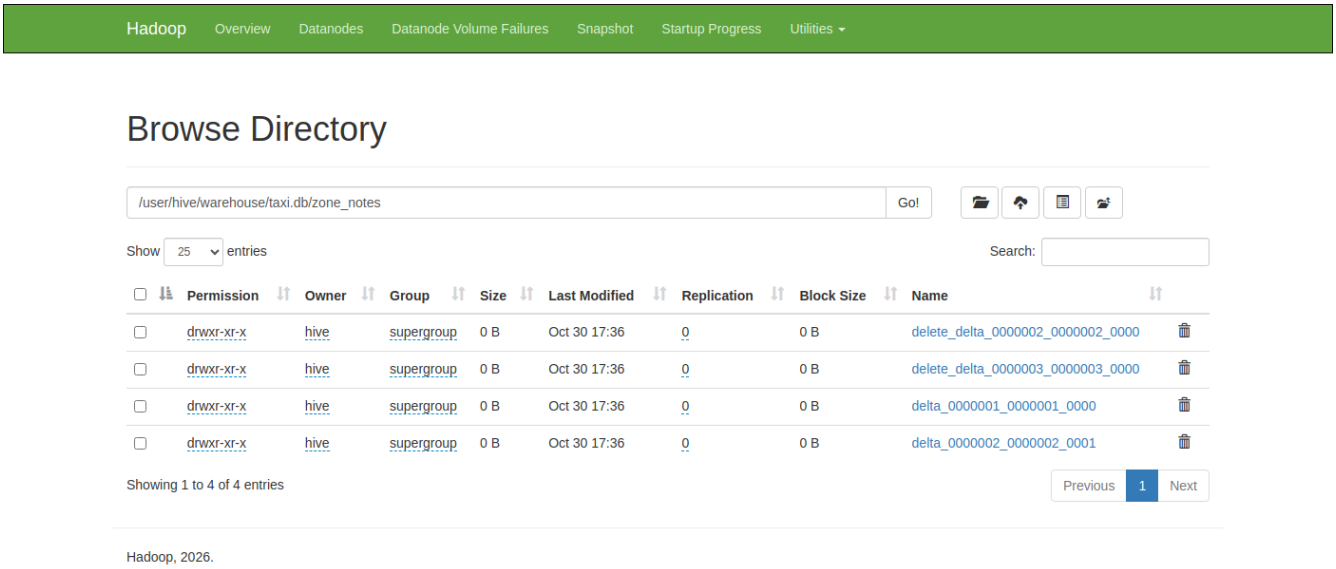


Stretch

ACID. `zone_notes` với `transactional=true` là `MANAGED_TABLE` thật. Sau INSERT, UPDATE rồi DELETE, HDFS có `delta_0000001` (insert), `delete_delta_0000002` + `delta_0000002` (UPDATE = xoá + ghi lại) và `delete_delta_0000003` (DELETE). Không file nào bị sửa tại chỗ; reader merge các delta khi đọc, compaction gộp chúng về sau.

Statistics. `ANALYZE ... COMPUTE STATISTICS FOR COLUMNS` mất 18.4 s. Sau đó, với `hive.compute.query.using.stats=true`, `count(*)` **vẫn chạy DAG Tez** (EXPLAIN vẫn có `TableScan`, ~1.6–2.6 s). Lý do: `trips_orc` thực chất là bảng **EXTERNAL** (translated), và Hive không tin stats của bảng external để trả lời thay cho việc quét, vì file có thể bị thay đổi ngoài Hive. Với bảng managed/ACID, câu trả lời sẽ lấy thẳng từ metastore (chỉ còn Fetch, không có job).


L03 — Stretch: statistics + ACID (delta / delete_delta)
$ stretch: statistics + ACID
+-------------+
| scan_count |
+-------------+
| 2964624 |
+-------------+
1 row selected (2.132 seconds)
+--------------+
| stats_count |
+--------------+
| 2964624 |
+--------------+
1 row selected (1.569 seconds)
3 rows affected (3.342 seconds)
1 row affected (4.34 seconds)
1 row affected (3.121 seconds)
+------------------------+-------------------+
| zone_notes.locationid | zone_notes.note |
+------------------------+-------------------+
| 132 | JFK Airport |
| 138 | LaGuardia |
+------------------------+-------------------+
2 rows selected (1.578 seconds)
Table Type:	MANAGED_TABLE	NULL
	transactional	true
	transactional_properties	default
$ hdfs dfs -ls -R .../zone_notes
zone_notes/delete_delta_0000002_0000002_0000
zone_notes/delete_delta_0000002_0000002_0000/bucket_00000_0
zone_notes/delete_delta_0000003_0000003_0000
zone_notes/delete_delta_0000003_0000003_0000/bucket_00000_0
zone_notes/delta_0000001_0000001_0000
zone_notes/delta_0000001_0000001_0000/bucket_00000_0
zone_notes/delta_0000002_0000002_0001
zone_notes/delta_0000002_0000002_0001/bucket_00000_0



Trả lời checkpoint

Q1. Metastore lưu gì, HDFS lưu gì? DROP TABLE thì sao? Metastore (bảng `TBLS`, `SDS`, `PARTITIONS`… trong Postgres): database, tên bảng, cột và kiểu, SerDe, input/output format, kiểu bảng, **LOCATION** của bảng và của từng partition, giá trị partition, thống kê (numRows, totalSize…), trạng thái ACID/transaction. **HDFS** chỉ chứa file dữ liệu. `DROP TABLE trips_csv` (external): chỉ xoá metadata, file trong `/data/taxi_csv` vẫn còn. `DROP TABLE trips_orc` (external + `external.table.purge=TRUE`): xoá metadata **và** thư mục `/user/hive/warehouse/taxi.db/trips_orc`.

Q3. Vì sao ORC/Parquet nhỏ hơn CSV 4–5 lần? (thực đo 5.2× và 4.2×) (a) **Lưu theo cột**: giá trị cùng kiểu, cùng cột nằm cạnh nhau nên các encoding dictionary, RLE, delta, bit-packing rất hiệu quả (`vendorid`, `payment_type`, `ratecodeid` chỉ có vài giá trị khác nhau). (b) **Kiểu nhị phân + nén** (ZLIB/Snappy) thay cho chữ số dạng text và dấu phân cách: một timestamp là 8 byte số thay vì chuỗi 19 ký tự. ORC + ZLIB nén mạnh hơn Parquet + Snappy (Snappy ưu tiên tốc độ hơn tỉ lệ nén).

Q4. Vì sao Q1 nhanh hơn trên ORC/Parquet? (8.53 s → 1.89 / 1.62 s, nhanh ~4.5–5×) * **Column pruning**: Q1 chỉ cần 2–3 trong 19 cột (`pulocationid`, `fare_amount`, cột partition). ORC/Parquet chỉ đọc các column chunk đó, còn CSV phải đọc và parse **toàn bộ** 274 MB, từng dòng. * **Partition pruning**: với ORC/Parquet, filter nằm trên `pickup_month`, nên Hive chỉ mở thư mục `pickup_month=2024-01` (đã chứng minh bằng EXPLAIN DEPENDENCY, 1 so với 5 partition). Bảng CSV không có partition nên phải quét hết rồi lọc theo timestamp. * **Predicate pushdown / min-max statistics**: mỗi stripe/row group lưu min/max và có row index, nên reader bỏ qua được các khối không thể khớp điều kiện. * Ngoài ra, reader ORC/Parquet được **vectorized** (xử lý theo batch 1,024 dòng) và không tốn công parse text

thành số.

Q6. Partition theo `pulocationid` + ngày? Theo `tpep_pickup_datetime`? `pulocationid` × ngày ≈ 265 × 31 ≈ **8,000 partition mỗi tháng**, mỗi partition chỉ có một file vài KB đến vài trăm KB. Đây là **small files problem**: NameNode phải giữ hàng chục nghìn object trong heap, **Metastore** phải lưu hàng nghìn dòng `PARTITIONS`/`SDS` (query planning chậm vì phải liệt kê và lấy metadata partition), và mỗi file nhỏ thành một split/task riêng nên overhead lấn át thời gian xử lý thật. Partition theo timestamp gốc thì gần như **mỗi giây một partition**: hàng triệu thư mục, file vài trăm byte, metastore và NameNode quá tải. Nguyên tắc: partition theo cột có cardinality thấp mà query hay lọc, mỗi partition nên chứa từ hàng trăm MB đến khoảng 1 GB.

Q7. Partitioning hay bucketing? Vì sao bucketing có số file cố định? Partitioning tạo thư mục theo **giá trị** cột, dùng cho cột cardinality thấp mà query hay lọc (ngày, tháng, quốc gia), để bỏ qua cả thư mục khi đọc. **Bucketing** chia dữ liệu theo `hash(col) mod N` vào N file, dùng cho cột cardinality cao (id, `pulocationid`). Nó cho phép sampling (`TABLESAMPLE`), bucket map join hoặc sort-merge-bucket join, và tránh shuffle khi hai bảng được bucket giống nhau trên khoá join. Số file cố định vì **N được chốt trong DDL** (`INTO 4 BUCKETS`): mỗi dòng đi vào đúng bucket `hash mod 4` bất kể dữ liệu nhiều hay ít (thực nghiệm: đúng 4 file, mọi dòng JFK nằm trong `000002_0`).

Q2. Trong file trips, `ROW FORMAT DELIMITED` (LazySimpleSerDe) chỉ tách theo dấu phẩy, đủ dùng vì không có ngoặc kép hay header. File zones có header và `"giá trị trong ngoặc kép"` (ví dụ `"Allerton/Pelham Gardens"`), nên cần `OpenCSVSerde`. Nhược điểm của nó: **mọi cột đều là STRING** (phải tự CAST, như `CAST(z.locationid AS INT)` trong Q2), parse chậm hơn, không xử lý NULL đặc biệt.

Q5. Partition tĩnh ghi giá trị ngay trong câu lệnh (`PARTITION (pickup_month='2024-01')`). Partition động lấy giá trị từ dữ liệu (cột cuối của SELECT). Mặc định là `strict` (bắt buộc ít nhất một giá trị tĩnh) để một câu query lỗi không vô tình tạo hàng nghìn partition, file và object trong metastore. Lab đã thấy điều này ở quy mô nhỏ: các dòng 2002/2009 sinh ra partition "lạ".

Q8. Có 18 dòng ngoài tháng 01/2024: 2 dòng 2002-12-31 (một cặp +10.5/−10.5, tức một chuyến bị hoàn tiền), 3 dòng 2009-01-01, 10 dòng 2023-12-31 23:39–23:58 (chuyến bắt đầu trước nửa đêm giao thừa) và 3 dòng 2024-02-01 00:00 (chuyến bắt đầu ngay sau nửa đêm). Các dòng năm 2002/2009 là lỗi đồng hồ thiết bị, còn các dòng 2023-12 và 2024-02 là chuyến thật nằm ở biên tháng. Trong pipeline thật: kiểm tra event time so với tháng của file nguồn; đưa dòng ngoài khoảng hợp lệ vào bảng **quarantine/rejected** kèm lý do và đếm/cảnh báo; các dòng biên tháng hợp lệ thì ghi vào đúng partition (hoặc giữ lại theo tháng của file); không để dynamic partitioning âm thầm tạo partition rác.